Ambiguous Grammars

```
expr ::= intLiteral | ident
       | expr + expr | expr / expr
```

ident + intLiteral / ident + ident

Each node in parse tree is given by one grammar alternative.

Ambiguous grammar: if some token sequence has **multiple parse trees** (then it is has multiple abstract trees).

Ambiguous Expression Grammar

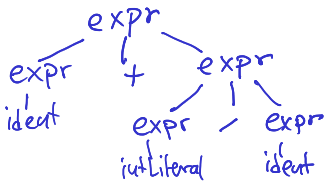
```
expr ::= intLiteral | ident
       | expr + expr | expr / expr
```

Example input:

ident + intLiteral / ident

has two parse trees, one suggested by

ident + intLiteral / ident



and one by

ident + intLiteral / ident

Suppose Division Binds Stronger

```
expr ::= intLiteral | ident
       | expr + expr | expr / expr
```

Example input:

ident + intLiteral / ident

has two parse trees, one suggested by

ident + intLiteral / ident

and one by **a bad tree**

ident + intLiteral / ident

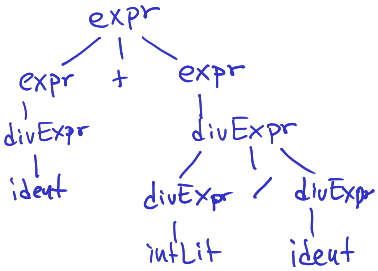
We do not want arguments of **/** expanding into expressions with **+** as the top level.

Layering the Grammar by Priorities

```
expr ::= intLiteral | ident
       | expr + expr | expr / expr
```

is transformed into a **new grammar**:

```
expr ::= expr + expr | divExpr
divExpr ::= intLiteral | ident
          | divExpr / divExpr
```



The bad tree

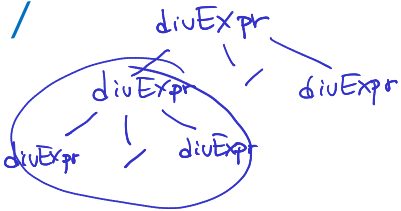
ident + intLiteral / ident

cannot be derived in the new grammar.

New grammar: same language, fewer parse trees!

Left Associativity of /

```
expr ::= expr + expr | divExpr
divExpr ::= intLiteral | ident
            | divExpr / divExpr
```

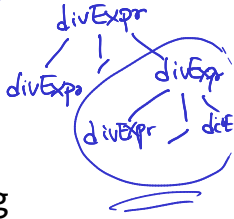


Example input:

ident / intLiteral / ident x/9/z

has two parse trees, one suggested by

ident / intLiteral / ident (x/9)/z



and one by **a bad tree**

ident / intLiteral / ident x/(9/z)

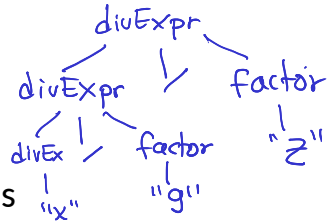
We do not want RIGHT argument of / expanding into expression with / as the top level.

Left Associativity - Left Recursion

x / g / z

```
expr ::= expr + expr | divExpr
divExpr ::= intLiteral | ident
            | divExpr / divExpr
```

```
expr ::= expr + expr | divExpr
divExpr ::= divExpr / factor
            | factor
factor ::= intLiteral | ident
```



No bad / trees

Still bad + trees

LR

```
expr ::= expr + divExpr | divExpr
divExpr ::= factor | divExpr / factor
factor ::= intLiteral | ident
```

No bad trees.

Left recursive!

Left vs Right Associativity

```
expr ::= expr + divExpr | divExpr
divExpr ::= factor | divExpr / factor
factor ::= intLiteral | ident
```

Left associative
Left recursive,
so not LL(1).

```
expr ::= divExpr + expr | divExpr
divExpr ::= factor | factor / divExpr
factor ::= intLiteral | ident
```

Unique trees.
Associativity wrong.
No left recursion.

3 (6 / 2)

```
expr ::= divExpr exprSeq
exprSeq ::= + expr | ε
divExpr ::= factor divExprSeq
divExprSeq ::= / divExpr | ε
factor ::= intLiteral | ident
```

Unique trees.
Associativity wrong.
LL(1): easy to pick an alternative to use.

Recursive Descent
LL(1) Parsing

- useful parsing technique
- to make it work, we might need to transform the grammar

Recursive Descent is Decent

Recursive *descent* is a *decent* parsing technique

- can be easily implemented manually based on the grammar (which may require transformation)
- efficient (linear) in the size of the token sequence

Correspondence between grammar and code

- concatenation → ;
- alternative (|) → if
- repetition (*) → while
- nonterminal → recursive procedure

A Rule of While Language Syntax

// Where things work very nicely for recursive descent!

statmt ::=

println (*stringConst* , *ident*)

| *ident* = *expr*

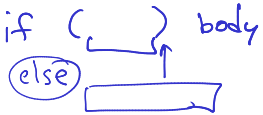
| *if* (*expr*) *statmt* (*else statmt*)$^{?}$

| *while* (*expr*) *statmt*

| { *statmt** }

Parser for the statmt (rule -> code)

```
def skip(t : Token) = if (lexer.token == t) lexer.next
  else error("Expected"+ t)
def statmt = {
  if (lexer.token == Println) { lexer.next;
    skip(openParen); skip(stringConst); skip(comma);
    skip(identifier); skip(closedParen)
  } else if (lexer.token == Ident) { lexer.next;
    skip(equality); expr
  } else if (lexer.token == ifKeyword) { lexer.next;
    skip(openParen); expr; skip(closedParen); statmt;
    if (lexer.token == elseKeyword) { lexer.next; statmt }
  // | while ( expr ) statmt
```



Continuing Parser for the Rule

// | while (expr) statmt

} **else if** (lexer.token == whileKeyword) { lexer.next;

skip(openParen); expr; skip(closedParen); statmt

e

while (e,s)

s

// | { statmt }*

} **else if** (lexer.token == openBrace) { lexer.next;

while (**isFirstOfStatmt**) { statmt }

skip(closedBrace)

} **else** { error("Unknown statement, found token " +
lexer.token) }

How to construct **if** conditions?

{if} }

statmt ::= println (stringConst , ident)
| if (expr) statmt (else statmt)?
| while (expr) statmt

- Look what each alternative starts with to decide what to parse
- Here: we have terminals at the beginning of each alternative
- More generally, we have 'first' computation, as for regular expressions
- Consider a grammar G and non-terminal N

first(L) = { a | ∃w. aw ∈ L }

$L_G(N)$ = { set of strings that N can derive }

e.g. L(statmt) – all statements of while language

first(N) = { a | aw in $L_G(N)$, a – terminal, w – string of terminals}

first(statmt) = { **println**, **ident**, **if**, **while**, **{** }

first(**while** (expr) statmt) = { **while** } - we will give an algorithm

Formalizing and Automating Recursive Descent: LL(1) Parsers

Task: Rewrite Grammar to make it suitable for recursive descent parser

- Assume the priorities of operators as in Java

```
expr ::= expr (+|-|*|/) expr
       | name | `(' expr `)'
name ::= ident
```

LISP
JSON
XML

Grammar vs Recursive Descent Parser

```
expr ::= term termList
termList ::= + term termList
           | - term termList
           | ε
term ::= factor factorList
factorList ::= * factor factorList
             | / factor factorList
             | ε
factor ::= name | ( expr )
name ::= ident
```

```
def expr = { term; termList }
def termList =
  if (token==PLUS) {
    skip(PLUS); term; termList
  } else if (token==MINUS)
    skip(MINUS); term; termList
  }
def term = { factor; factorList }
...
def factor =
  if (token==IDENT) name
  else if (token==OPAR) {
   skip(OPAR); expr; skip(CPAR)
  } else error("expected ident or )")
```

term
| term + termList

Note that the abstract trees we would create in this example do not strictly follow parse trees.

Rough General Idea

$$
\begin{array}{l}
A ::= B_1 \ldots B_p \\
\quad\;\; \mid C_1 \ldots C_q \\
\quad\;\; \mid D_1 \ldots D_r
\end{array}
$$

```
def A =
  if (token ∈ T1) {
    B1 ... Bp
  else if (token ∈ T2) {
    C1 ... Cq
  } else if (token ∈ T3) {
    D1 ... Dr
  } else error("expected T1,T2,T3")
```

where:

$T1 = \textbf{first}(B_1 \ldots B_p)$

$T2 = \textbf{first}(C_1 \ldots C_q)$

$T3 = \textbf{first}(D_1 \ldots D_r)$

$\textbf{first}(B_1 \ldots B_p) = \{a \in \Sigma \mid B_1 \ldots B_p \Rightarrow \ldots \Rightarrow aw\}$

T1, T2, T3 should be **disjoint** sets of tokens.

Computing **first** in the example

```
expr ::= term termList
termList ::= + term termList
           | - term termList
           | ε
term ::= factor factorList
factorList ::= * factor factorList
             | / factor factorList
             | ε
factor ::= name | ( expr )
name ::= ident
```

first(name) = {**ident**}

first(**(** expr **)**) = { **(** }

first(factor) = first(name)

U first(**(** expr **)**)

= {**ident**} U{ **(** }

= {**ident**, **(** }

first(* factor factorList) = { * }

first(/ factor factorList) = { / }

first(factorList) = { *, / }

first(term) = first(factor) = {**ident**, **(** }

first(termList) = { **+** , **-** }

first(expr) = first(term) = {**ident**, **(** }

Algorithm for **first**: Goal

Given an arbitrary context-free grammar with a set of rules of the form $X ::= Y_1 \ldots Y_n$ compute first for each right-hand side and for each symbol.

How to handle

- alternatives for one non-terminal
- sequences of symbols
- nullable non-terminals
- recursion

Rules with Multiple Alternatives

$$A ::= B_1 \ldots B_p \mid C_1 \ldots C_q \mid D_1 \ldots D_r$$

$$\text{first}(A) = \text{first}(B_1 \ldots B_p) \cup \text{first}(C_1 \ldots C_q) \cup \text{first}(D_1 \ldots D_r)$$

Sequences

$$\text{first}(B_1 \ldots B_p) = \text{first}(B_1)$$ if not nullable(B_1)

$$\text{first}(B_1 \ldots B_p) = \text{first}(B_1) \cup \ldots \cup \text{first}(B_k)$$

if nullable(B_1), ..., nullable(B_{k-1}) and

not nullable(B_k) or k=p

$B_1 \ldots B_{k-1} B_k \ldots B_p$

Abstracting into Constraints

recursive grammar: constraints over finite sets: expr' is first(expr)

```
expr ::= term termList
termList ::= + term termList
           | - term termList
           | ε
term ::= factor factorList
factorList ::= * factor factorList
             | / factor factorList
             | ε
factor ::= name | ( expr )
name ::= ident
```

```
expr' = term'
termList' = {+}
          U {-}

term' = factor'
factorList' = {*}
            U { / }

factor' = name' U { ( }
name' = { ident }
```

nullable: termList, factorList

For this nice grammar, there is no recursion in constraints. Solve by substitution.

Example to Generate Constraints

S ::= X | Y
X ::= **b** | S Y
Y ::= Z X **b** | Y **b**
Z ::= ε | **a**

S' = X' U Y'
X' = {b} U S'
Y' = Z' U X' U Y'
Z' = {a}

terminals: **a**,**b**
non-terminals: S, X, Y, Z

reachable (from S):
productive:
nullable: Z

First sets of terminals:
S', X', Y', Z' ⊆ {a,b}

Example to Generate Constraints

S ::= X | Y
X ::= **b** | S Y
Y ::= Z X **b** | Y **b**
Z ::= ε | **a**

$$S' = X' \cup Y'$$
$$X' = \{b\} \cup S'$$
$$Y' = Z' \cup \mathbf{X'} \cup Y'$$
$$Z' = \{a\}$$

terminals: **a**,**b**
non-terminals: S, X, Y, Z

reachable (from S): S, X, Y, Z
productive: X, Z, S, Y
nullable: Z

These constraints are recursive.
How to solve them?

$$S', X', Y', Z' \subseteq \{a,b\}$$

How many candidate solutions

- in this case?
- for k tokens, n nonterminals?

Iterative Solution of **first** Constraints

	S'	X'	Y'	Z'
1.	{}	{}	{}	{}
2.	{}	{b}	{b}	{a}
3.	{b}	{b}	{a,b}	{a}
4.	{a,b}	{a,b}	{a,b}	{a}
5.	{a,b}	{a,b}	{a,b}	{a}

S' = X' U Y'
X' = {b} U S'
Y' = Z' U **X'** U Y'
Z' = {a}

- Start from all sets empty.
- Evaluate right-hand side and assign it to left-hand side.
- Repeat until it stabilizes.

Sets grow in each step

- initially they are empty, so they can only grow
- if sets grow, the RHS grows (U is monotonic), and so does LHS
- they cannot grow forever: in the worst case contain all tokens